

# ESP32 PlatformIO Project — Secure Password Manager (Arduino + ESP-IDF)

This PlatformIO project template enables using **Arduino + ESP-IDF** together and provides secure AES using an efuse (BLOCK3) key and hardware AES engine. It includes build files and example code.

---

## Included files (generated)

- `platformio.ini` — PlatformIO configuration
  - `src/main.cpp` — Arduino-style sketch that uses ESP-IDF APIs (`esp_aes`, `esp_efuse`) to perform AES using efuse key slot 3
  - `lib/secure_sd/secure_sd.h` — header for secure SD encrypted storage
  - `lib/secure_sd/secure_sd.cpp` — implementation
  - `README.md` — setup & burn key instructions
- 

## Important: Uploaded screenshot (for reference)

Local path to your uploaded screenshot (will be transformed to URL by the environment):

```
/mnt/data/212e50b1-9a11-41f1-af4d-027adbd5f386.png
```

---

## platformio.ini

```
[env:esp32dev]
platform = espressif32
board = esp32dev
framework = arduino, espidf
monitor_speed = 115200
build_flags =
    -D ARDUINO_IDF_COMPAT=1
    -D ESP32
; increase optimization for crypto
build_unflags = -Og

; keep default upload protocol

lib_deps =
    paulstoffregen/SD@^1.2.4
```

```
[platformio]
default_envs = esp32dev
```

## src/main.cpp

```
#include <Arduino.h>
extern "C" {
#include "esp_system.h"
#include "esp_log.h"
#include "esp_efuse.h"
#include "esp_efuse_table.h"
#include "esp_aes.h"
}

#include "secure_sd.h"

static const char* TAG = "secure_mgr";

void setup() {
    Serial.begin(115200);
    delay(1000);
    Serial.println("Secure Password Manager – AES with efuse BLOCK3 demo");

    // Initialize SD and secure storage
    if (!secure_sd_begin()) {
        Serial.println("Secure SD init failed");
        while (1) delay(1000);
    }

    // Example: encrypt and decrypt a 16-byte block using BLOCK3 key
    const uint8_t plaintext[16] = "HelloWorld123456"; // 16 bytes
    uint8_t ciphertext[16];
    uint8_t decrypted[16];

    if (!secure_aes_encrypt_block(plaintext, ciphertext, sizeof(plaintext))) {
        Serial.println("Encryption failed");
    } else {
        Serial.print("Ciphertext: ");
        for (int i=0; i<16; i++) Serial.printf("%02X", ciphertext[i]);
        Serial.println();

        if (!secure_aes_decrypt_block(ciphertext, decrypted, sizeof(ciphertext))) {
            Serial.println("Decryption failed");
        }
    }
}
```

```

    } else {
        Serial.print("Decrypted: ");
        for (int i=0;i<16;i++) Serial.print((char)decrypted[i]);
        Serial.println();
    }
}
}

void loop() {
    delay(10000);
}

```

## lib/secure\_sd/secure\_sd.h

```

#pragma once
#include <Arduino.h>

bool secure_sd_begin();

bool secure_aes_encrypt_block(const uint8_t *in, uint8_t *out, size_t len);
bool secure_aes_decrypt_block(const uint8_t *in, uint8_t *out, size_t len);

```

## lib/secure\_sd/secure\_sd.cpp

```

#include "secure_sd.h"
extern "C" {
#include "esp_aes.h"
#include "esp_efuse.h"
#include "esp_efuse_table.h"
}

#include <Arduino.h>

// Uses efuse block 3 (EFUSE_BLK_KEY3)
#define KEY_BLOCK EFUSE_BLK_KEY3

bool secure_sd_begin() {
    // Initialize SD, SPI, etc. (left to the user)
    // This function simply checks efuse KEY is present (usage)
    // Note: if read-protected, raw read may return zeros; that's expected.
    return true;
}

```

```

}

bool secure_aes_encrypt_block(const uint8_t *in, uint8_t *out, size_t len) {
    if (len % 16 != 0) return false;

    esp_aes_context_t ctx;
    esp_aes_init(&ctx);
    esp_err_t err = esp_aes_setkey_efuse(&ctx, ESP_AES_ENCRYPT,
    ESP_EFUSE_KEY_PURPOSE_USER, KEY_BLOCK);
    if (err != ESP_OK) {
        esp_aes_free(&ctx);
        return false;
    }

    uint8_t iv[16] = {0}; // For demo only – use random IV stored with ciphertext
    in production
    err = esp_aes_crypt_cbc(&ctx, ESP_AES_ENCRYPT, len, iv, in, out);
    esp_aes_free(&ctx);
    return (err == ESP_OK);
}

bool secure_aes_decrypt_block(const uint8_t *in, uint8_t *out, size_t len) {
    if (len % 16 != 0) return false;

    esp_aes_context_t ctx;
    esp_aes_init(&ctx);
    esp_err_t err = esp_aes_setkey_efuse(&ctx, ESP_AES_DECRYPT,
    ESP_EFUSE_KEY_PURPOSE_USER, KEY_BLOCK);
    if (err != ESP_OK) {
        esp_aes_free(&ctx);
        return false;
    }

    uint8_t iv[16] = {0};
    err = esp_aes_crypt_cbc(&ctx, ESP_AES_DECRYPT, len, iv, in, out);
    esp_aes_free(&ctx);
    return (err == ESP_OK);
}

```

## README.md (setup & burning key)

# Secure Password Manager – Setup

1. Generate a 32-byte AES key on your PC and save as aes\_key.bin.

2. Burn the key into BLOCK3 (only once):

```
espefuse --port COM5 --chip esp32 burn_key BLOCK3 aes_key.bin
```

3. Set purpose to USER (if needed):

```
espefuse --port COM5 --chip esp32 set_key_purpose BLOCK3 USER
```

4. Write-protect and read-protect (permanent):

```
espefuse --port COM5 --chip esp32 write-protect-efuse BLOCK3  
espefuse --port COM5 --chip esp32 read-protect-efuse BLOCK3
```

5. Now the key is usable by the hardware AES engine and will not be readable by software.

6. Build and upload the PlatformIO project.

---

## Notes & Caveats

- This template uses `esp_aes_setkey_efuse()` and `esp_aes_crypt_cbc()` from ESP-IDF.
- IV handling, authentication (e.g., AES-GCM or HMAC) and secure random IV storage must be implemented before production use.
- Always back up your plaintext key securely before burning — efuse write-protect is permanent.

---

If you want, I can now:

1. Create the actual PlatformIO ZIP with these files and provide a download link.
2. Walk you through `espefuse` commands step-by-step on Windows to burn and protect the key safely.
3. Expand the secure SD module to include per-entry IVs, MACs, and on-disk structure.

Tell me which next step you prefer.